# Table of Contents

*Pipelined Datapath and Control*
*Data Hazards: Forwarding vs Stalling*
*Control Hazards*
*Conclusion*

”

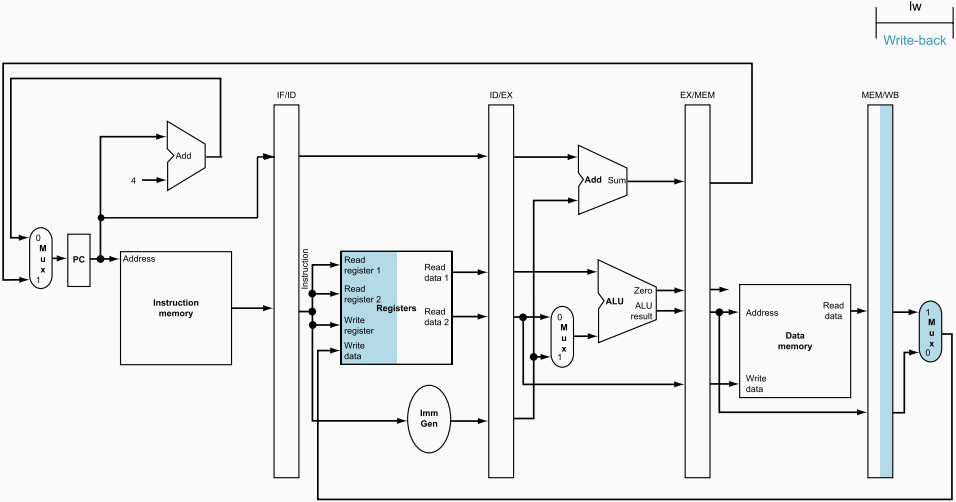
What do you mean, why’s it got to be built? It’s a bypass. You’ve got to build bypasses.

*The Hitchhiker’s Guide to the Galaxy, 1979*

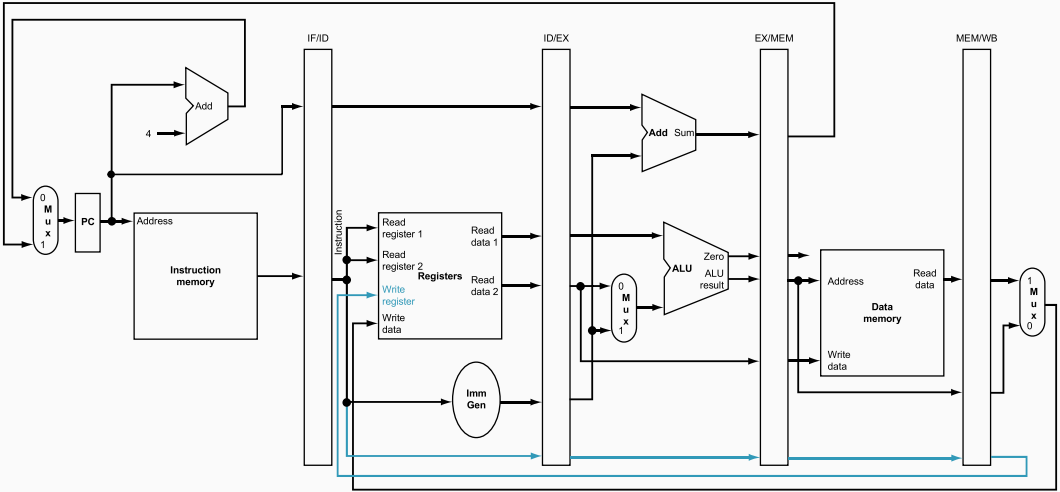
Douglas Adams

# Pipelined Datapath and Control

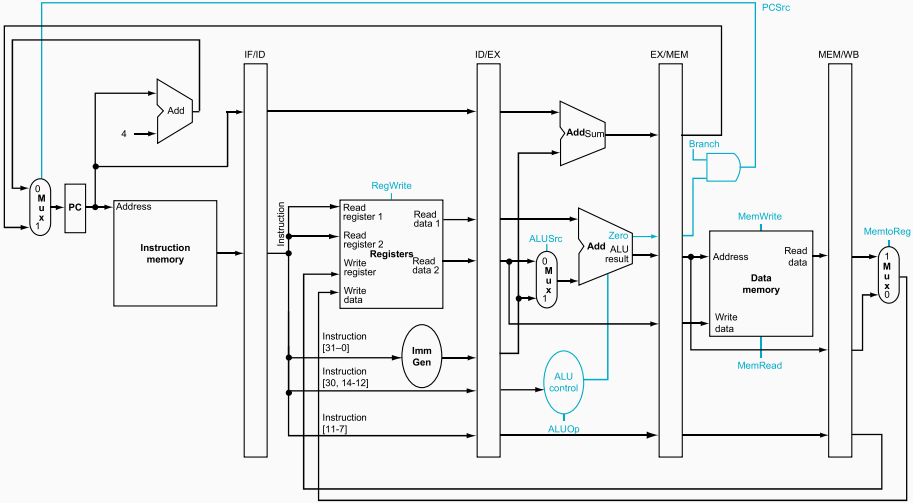
# WB for Load (Buggy)



# WB for Load (Corrected)



# Pipelined Control Signals (Simplified)



# Control Grouping by Stage

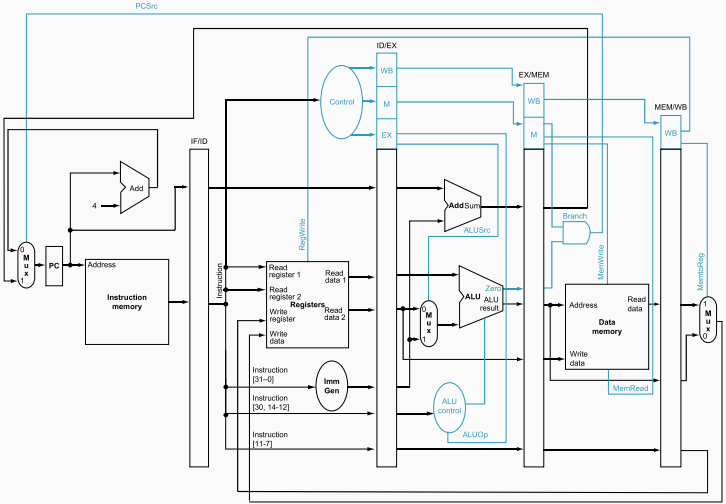
Control lines are divided into groups according to their active pipeline stage —

1. *IF:* Signals to read memory and write the *PC* are always asserted
2. *ID:* Source registers are in fixed locations; no special control needed for reading
3. *EX:* Sets ALUOp and ALUSrc to select the operation and the second ALU operand (Register vs Immediate)
4. *MEM:* Sets Branch, MemRead, and MemWrite; PCSrc is determined by the Branch signal combined with the ALU Zero result
5. *WB:* Sets MemtoReg to choose between ALU result or memory data and RegWrite to commit the chosen value to the register file

# Implementing Pipelined Control

- Control information is created during the Instruction Decode stage
- Passing Control: Control signals are passed down the pipeline by extending the pipeline registers
- As an instruction moves, its backpack of control signals moves with it to the appropriate stage
- This design ensures the signals reach the components at the exact moment the instruction arrives

# Pipelined Datapath with Control



# Data Hazards

# Detecting the Need to Forward

- Pass register numbers along pipeline (with values), e.g., ID/EX.RegisterRs1 → register number for Rs1 sitting in the ID/EX pipeline register
- Following this notation, ALU operand register numbers in EX stage are given by ID/EX.RegisterRs1 and ID/EX.RegisterRs2
- We get data hazards when —
  1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1
  1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2
  2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1
  2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2
- But only if forwarding instruction will write to a register, i.e., EX/MEM.RegWrite or MEM/WB.RegWrite is asserted
- And only if Rd for that instruction is not `x0`, i.e., EX/MEM.RegisterRD $\neq 0$, MEM/WB.RegisterRD $\neq 0$

# Dependency Detection Example

## Example

- For the previous example code sequence:

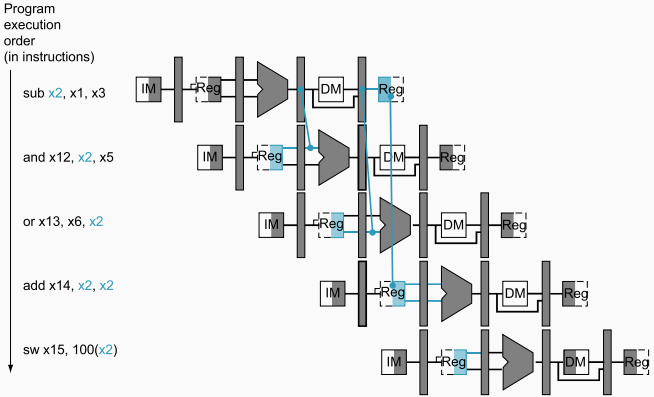
```
sub x2, x1, x3 // Register x2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sw x15, 100(x2) // Base (x2) depends on sub
```

- Respective hazard conditions are:
  - The `sub-and` is a type 1a hazard (EX/MEM.RegisterRd = ID/EX.RegisterRs1)
  - The `sub-or` is a type 2b hazard (MEM/WB.RegisterRd = ID/EX.RegisterRs2)
  - The two dependencies on the `sub-add` are not hazards because the register file supplies the proper data during the ID stage of `add`
  - There is no data hazard between `sub` and `sw` because `sw` reads `x2` the clock cycle after `sub` writes `x2`

# Dependencies after Forwarding

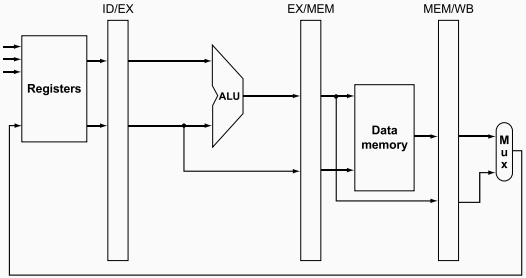
Time (in clock cycles) →

| | CC 1 | CC 2 | CC 3 | CC 4 | CC 5 | CC 6 | CC 7 | CC 8 | CC 9 |
|---|---|---|---|---|---|---|---|---|---|
| value of register x2: | 10 | 10 | 10 | 10 | 10/–20 | –20 | –20 | –20 | –20 |

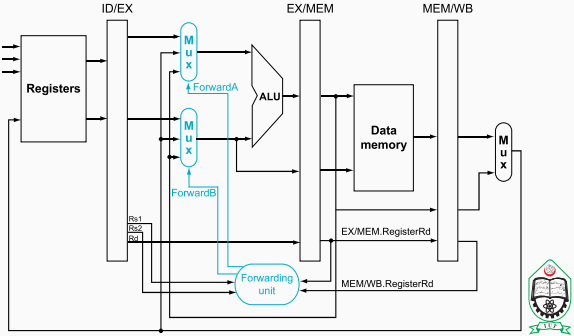


# Forwarding Paths

(*The diagrams have been simplified, such as, omission of immediate generation unit)




a. No forwarding




b. With forwarding

# Written Conditions for Detecting Hazards (EX Hazards)

```
if  (EX/MEM.RegWrite and
     (EX/MEM.RegisterRd ≠ 0) and
     (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
         ForwardA = 10
if  (EX/MEM.RegWrite and
     (EX/MEM.RegisterRd ≠ 0) and
     (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
         ForwardB = 10
```

# Written Conditions for Detecting Hazards (MEM Hazards)

```
if  (MEM/WB.RegWrite and
     (MEM/WB.RegisterRd ≠ 0) and
     (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
         ForwardA = 01
if  (MEM/WB.RegWrite and
     (MEM/WB.RegisterRd ≠ 0) and
     (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
         ForwardB = 01
```

# Double Data Hazard

- Consider the following code sequence:

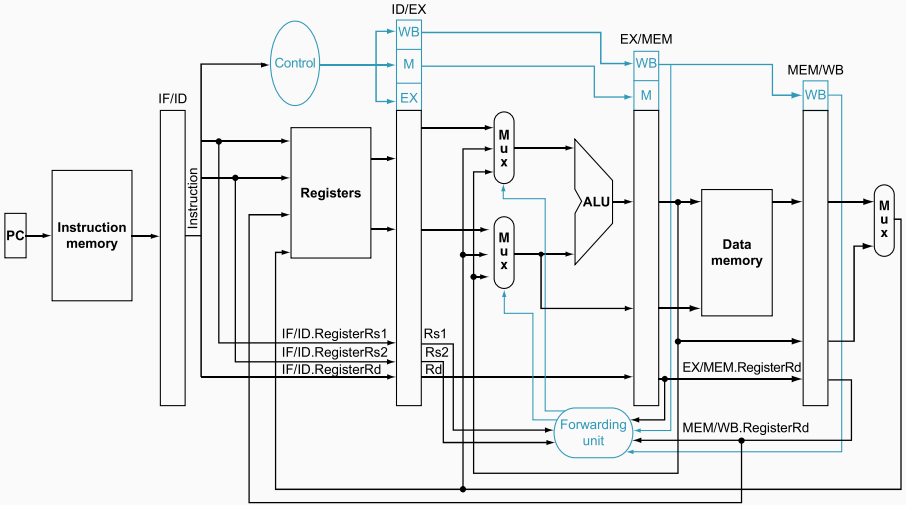
```
add x1, x1, x2
add x1, x1, x3
add x1, x1, x4
```

- Both hazards occur; we want to use the most recent
- Revise MEM hazard condition to only forward if EX hazard condition is not true

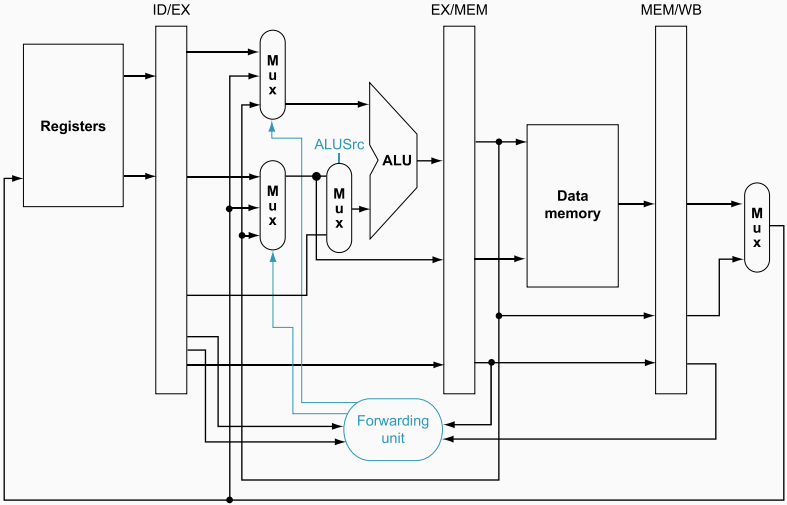
# Revised MEM Hazard Forwarding Condition

```
if  (MEM/WB.RegWrite and
     (MEM/WB.RegisterRd ≠ 0) and
     not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
         and (EX/MEM.RegisterRd = ID/EX.RegisterRs1)) and
     (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
         ForwardA = 01
if  (MEM/WB.RegWrite and
     (MEM/WB.RegisterRd ≠ 0) and
     not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
         and (EX/MEM.RegisterRd = ID/EX.RegisterRs2)) and
     (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
         ForwardB = 01
```

# Datapath with Forwarding



# Forwarding and Signed Immediate with a 2:1 Multiplexor



# Load-Use Hazard Detection

- Check when the instruction is decoded in the ID stage
- ALU operand register numbers in the ID stage are given by IF/ID.RegisterRs1 and IF/ID.RegisterRs2
- Load-use hazard condition:

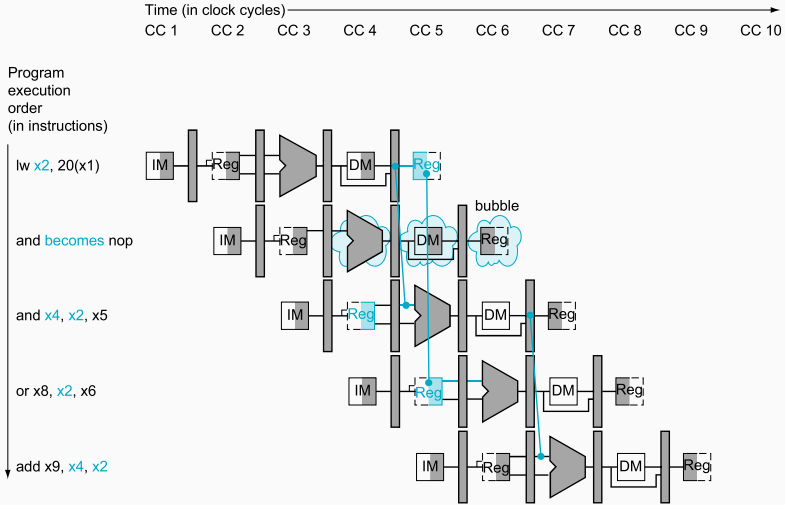
```
if  (ID/EX.MemRead and
    ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or
        (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
    // stall the pipeline
```

- If detected, stall and insert bubble

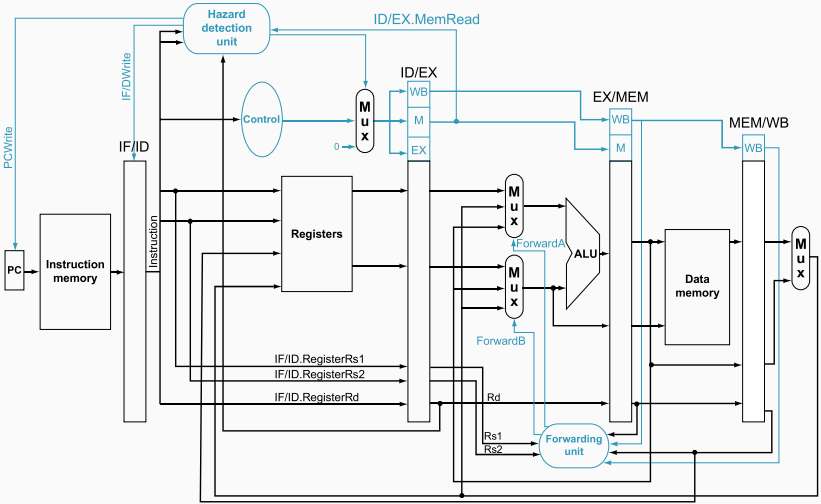
# How to Stall the Pipeline

- Force control values in the ID/EX register to 0 (zero)
  - EX, MEM, and WB do nop (no-operation)
- Prevent the update of *PC* and IF/ID register
  - Used instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows MEM to read data for `lw`
  - Can subsequently forward to the EX stage

# Stalling for Load-Use Data Hazard



# Datapath with Hazard Detection and Forwarding



# Control Hazards

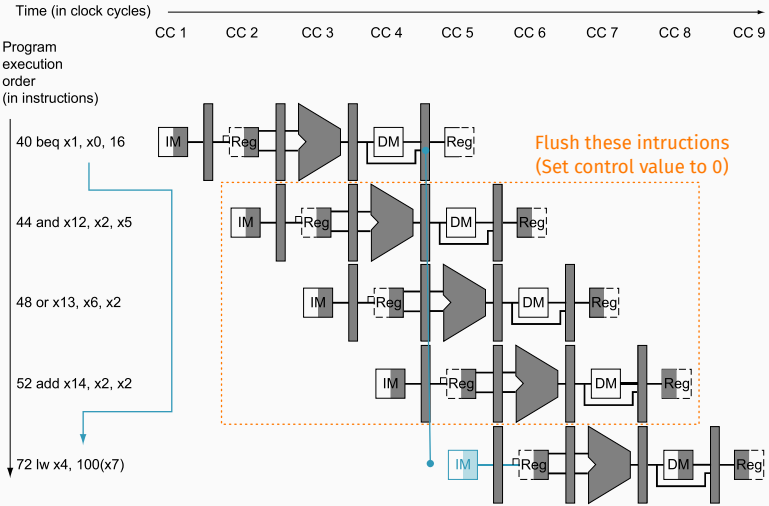
# Control Hazards Overview

- Hazards involving conditional branches are distinct from data hazards
- An instruction must be fetched every clock cycle to sustain the pipeline
- In standard design, branch decision occurs in the MEM pipeline stage
- Delay in determining the proper instruction to fetch is a control hazard (or branch hazard).
- Comparison to data hazards:
  - Simpler to understand
  - Occur less frequently
  - Forwarding is not directly effective against them

# Discarding Instructions (Flushing)

- Discarding involves changing original control values to 0s
- Similar to stalling for load-use data hazards, but more extensive
- When branch reaches MEM stage, must flush instructions in:
  1. IF stage
  2. ID stage
  3. EX stage
- In this case, the cost is delay by three stages

# Example Multi-Cycle Diagram of Control Hazard



# Optimization: Reducing Branch Delay

- Goal: Reduce cost of a taken branch
- Method: Move conditional branch execution earlier in pipeline
- Move from MEM stage to ID stage
- Required actions in ID:
  - Compute branch target address
  - Evaluate branch decision

# Moving Logic to ID Stage

- Address Calculation:
  - Easy to move
  - Move branch adder from EX to ID
  - Uses PC value and immediate field from IF/ID register
- Branch Decision:
  - Harder to implement (dependency)
  - Equality test: XOR individual bit positions of two registers and OR the resultant bits
  - Zero output means registers are equal

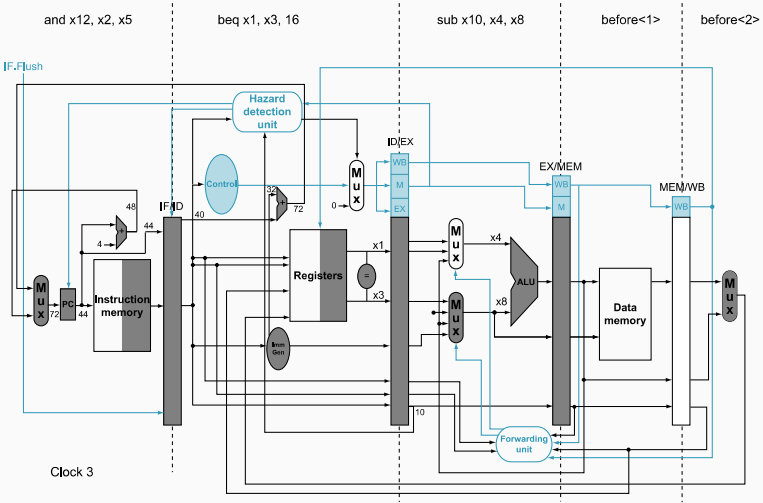
# Complications: Forwarding and Hazards

- Moving branch test to ID implies additional hardware
- Complication 1: Forwarding
  - Must decode, decide bypass, and test equality within ID
  - Requires new forwarding logic (distinct from ALU forwarding)
  - Operands may come from EX/MEM or MEM/WB registers
- Complication 2: Data Hazards
  - ALU instruction immediately preceding branch: Requires 1 stall
  - Load immediately preceding branch: Requires 2 stalls

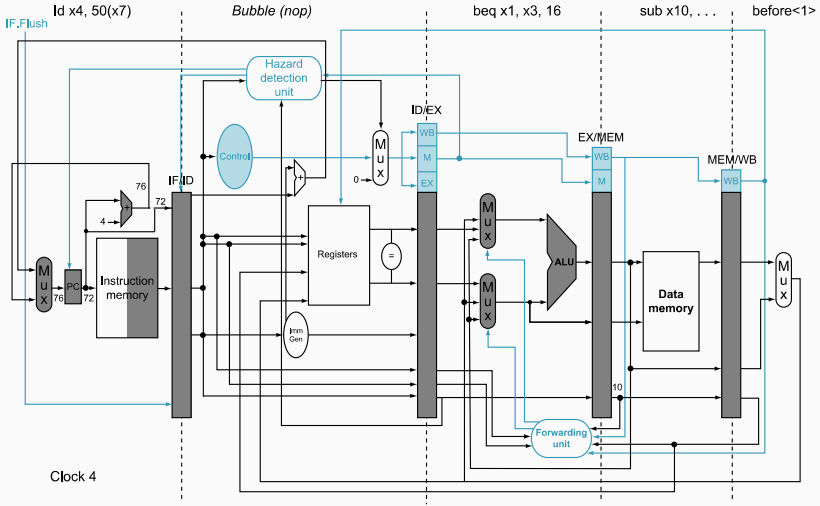
# Implementation and Results

- Moving execution to ID reduces taken branch penalty to one instruction (the one currently being fetched).
- Flushing mechanism:
  - Add control line IF.Flush
  - Make zero the instruction field of IF/ID pipeline register (instruction becomes `add x0, x0, x0`)
  - Transform fetched instruction into a **nop** (no operation)

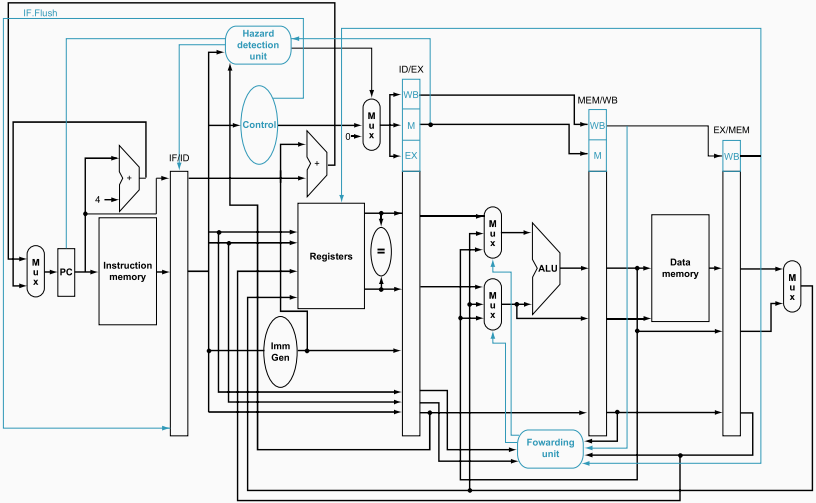
# Example: Branch Taken



# Example: Branch Taken



# Final Datapath and Control



# Dynamic Branch Prediction

- Static prediction (Assume Not Taken) is adequate for simple pipelines
- Performance issues arise with:
  - Deeper pipelines: Branch penalty increases in clock cycles
  - Multiple-issue pipelines: Branch penalty increases in instructions lost
- Aggressive pipelines require dynamic branch prediction
- Concept: Look up instruction address to check past behavior

# Branch Prediction Buffer (1-Bit Scheme)

- Also called a branch history table
- Small memory indexed by lower portion of branch instruction address
- Contains a bit indicating if branch was recently taken
- Mechanism:
  - Uses prediction as a hint for fetching
  - When hint is wrong: delete instructions, invert prediction bit, fetch proper sequence

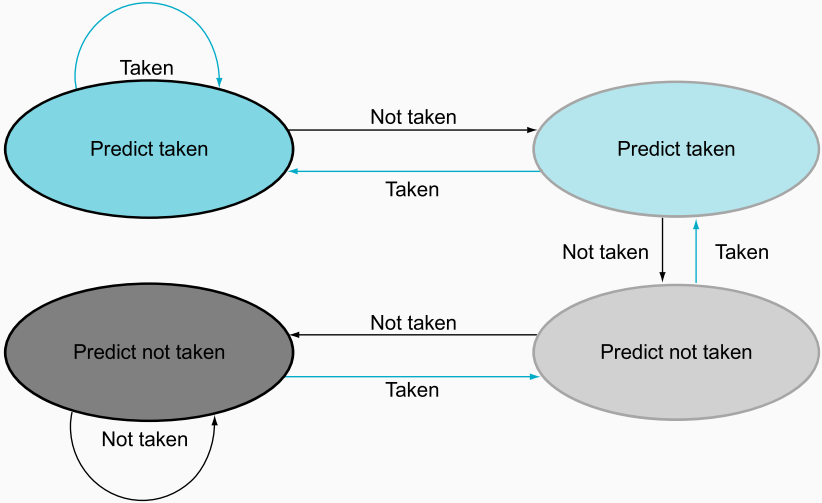
# Shortcoming of 1-Bit Prediction

- Performance issue – can mispredict twice for a branch that is almost always taken
- Example Scenario: Nested loop, inner loop branches 9 times, not taken once (exiting)
- Mispredictions:
  - Last iteration: Inevitable (predicts taken, actually not taken).
  - First iteration: Bit was flipped on previous exit (predicts not taken, actually taken).
- Result: 80% accuracy for a branch taken 90% of the time.

# 2-Bit Prediction Scheme

- Solution to the 1-bit oscillation problem
- Prediction must be wrong twice before it is changed
- Implementation:
  - Finite-state machine
  - Accessed with instruction address during IF stage
- If predicted taken, fetching begins as soon as PC is known (potentially ID stage)

# The States and Transitions in a 2-Bit Prediction Scheme



# Branch Target Buffer

- Prediction tells *direction*, but target address calculation still required
- Calculation takes one cycle in 5-stage pipeline (one-cycle penalty)
- Optimization: Use a Branch Target Buffer
- Acts as a cache to hold:
  - Destination program counter, or
  - Destination instruction

BTB enables branch address to be known in the IF stage, leading to no flushing required of the instruction in the IF stage when branch is calculated in the ID stage

# Correlating Predictors

- 2-bit scheme only uses local branch information
- Correlating Predictors: Combine local branch info with global behavior of recently executed branches
- Implementation:
  - Two 2-bit predictors per branch
  - Selection based on whether last executed branch was taken/not taken
- Global behavior acts as additional index-bits

This method takes an addditional global history register (GHR). The GHR stores the outcome of recent branches and uses that history in prediction of the currrent branch

# Tournament Predictors

- Use multiple predictors and track the best performer for each branch
- Typical structure:
  - One predictor based on local information
  - One predictor based on global behavior
- Selector:
  - Chooses which predictor to use
  - Operates like a counter, favoring the more accurate predictor
- Common in recent microprocessors.

Thank You!